PiCam for Raspberry-Pi-5 Running Ubuntu

Prerequisites: Raspberry Pi 5, with Ubuntu Terminal 24.04.3 LTS installed + ROS Jazzy (Barebones) Installed

---

Getting the Raspberry Pi Camera Module 3 (IMX708) to work on a Pi 5 with Ubuntu 24.04 and ROS 2 Jazzy is a bit trickier than on Raspberry Pi OS.

```
sudo nano /boot/firmware/config.txt
```

Scroll to the bottom and make sure the following line exists (add it if it's missing). This loads the driver for the PiCam 3. Note, if you're plugged into cam port 1, use 'cam1' below instead of 'cam0'

```
dtoverlay=imx708,cam0
```

Also, whilst you're at it, find this line…. 'camera_auto_detect=1' and change to…

```
# Disable auto-detect to avoid confusion
camera_auto_detect=0
```

Now reboot…

```
sudo reboot
```

Now we need to install all the necessary components for the camera to work on Ubuntu Terminal

```
sudo apt update
```

```
sudo apt install -y git python3-pip python3-yaml python3-ply
python3-jinja2 \ libyaml-dev libgnutls28-dev openssl libglib2.0-
dev build-essential meson ninja-build \ python3-colcon-meson
python3-colcon-common-extensions libgstreamer1.0-dev libgstreamer-
plugins-base1.0-dev gstreamer1.0-plugins-base gstreamer1.0-
plugins-good gstreamer1.0-plugins-bad gstreamer1.0-plugins-ugly
gstreamer1.0-libav gstreamer1.0-tools
```

Now we will clone the libcamera package into our RaspberryPi, in the home directory!

```
cd ~
git clone https://github.com/raspberrypi/libcamera.git cd
libcamera
```

Now run this command to configure and compile it…

```
meson setup build --buildtype=release -Dpipelines=rpi/vc4,rpi/pisp
-Dipas=rpi/vc4,rpi/pisp -Dv4l2=enabled -Dgstreamer=enabled -
Dtest=false
```

Check the output: Look at the bottom of the log this time. You want to see: Build targets in project: [number] (usually around 130-150) And **no** red ERROR messages about dependencies.

Next Steps: Compile and Install

Now that the configuration is correct, run the following commands in order:

1. Compile the library

This will take a few minutes on the Pi 5.

```
cd ~/libcamera
ninja -C build
```

2. Install to the system

```
sudo ninja -C build install
```

3. Update the system library cache

Crucial step. This tells Linux where to look for the new library you just created.

```
sudo ldconfig
```

4. Hardware Check (The Moment of Truth)

Once the above steps are done, verify that libcamera can actually "talk" to your IMX708 sensor:

```
cam -l
```

Success criteria: You should see output similar to:

[0:01:38.474240819] [3275] INFO Camera camera_manager.cpp:330 libcamera v0.5.2+99-bfd68f78
[0:01:38.504797672] [3278] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 20-11-2025 (11:04:51)

[0:01:38.623669551] [3278] INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /usr/local/share/libcamera/ipa/rpi/pisp/imx708.json
[0:01:38.637450375] [3278] INFO Camera camera_manager.cpp:220 Adding camera '/base/axi/pcie@120000/rp1/i2c@88000/imx708@1a' for pipeline handler rpi/pisp
[0:01:38.637478030] [3278] INFO RPI pisp.cpp:1179 Registered camera /base/axi/pcie@120000/rp1/i2c@88000/imx708@1a to CFE device /dev/media0 and ISP device /dev/media1 using PiSP variant BCM2712_C0
Available cameras:
1: External camera 'imx708' (/base/axi/pcie@120000/rp1/i2c@88000/imx708@1a)

If you see the camera listed there, **you are ready to build the ROS driver**.

Step 4: Build the ROS 2 Driver (camera_ros)

Now we build the ROS node that wraps the library you just installed. We **cannot** use sudo apt install ros-jazzy-camera-ros because that binary is linked against the wrong (standard Ubuntu) libcamera.

Also note, unlike the ldlidar_ros2 module, which we installed in its own workspace (because once compiled once it won't need compiling again) we will put the camera_ros module into our 'slambot_ws2' workspace as we will be fiddling with parameters etc, so it will need regularly recompiling.

1. Go to the slambot workspace SRC directory (if you haven't already):

   ```
   cd ~/ros_workspaces/slambot_ws2/src
   ```

2. Clone the camera_ros package:

   ```
   git clone https://github.com/christianrauch/camera_ros.git
   ```

3. Install dependencies (ignoring libcamera since we installed it manually):

   ```
   cd ~/ros_workspaces/slambot_ws2rosdep install --from-paths src --ignore-src -y --skip-keys=libcamera
   ```

4. Build the workspace:

   ```
   colcon build -packages-select camera_ros --symlink-install
   ```

Step 5: Run and Test

Everything is now ready to run.

1. Source your workspace:

   ```
   source ~/.bashrc
   ```

2. Run the camera node, specifying the port:

   ```
   ros2 run camera_ros camera_node --ros-args -p camera:=0
   ```

   You should see logs indicating the camera has been found and streams are configuring. Example below:

[0:25:06.872158064] [6766] INFO Camera camera_manager.cpp:330 libcamera v0.5.2+99-bfd68f78

[0:25:06.895501491] [6792] INFO RPI pisp.cpp:720 libpisp version v1.2.1 981977ff21f3 20-11-2025 (11:04:51)

[0:25:06.982542732] [6792] INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /usr/local/share/libcamera/ipa/rpi/pisp/imx708.json

[0:25:06.994035186] [6792] INFO Camera camera_manager.cpp:220 Adding camera '/base/axi/pcie@120000/rp1/i2c@88000/imx708@1a' for pipeline handler rpi/pisp

[0:25:06.994067888] [6792] INFO RPI pisp.cpp:1179 Registered camera /base/axi/pcie@120000/rp1/i2c@88000/imx708@1a to CFE device /dev/media0 and ISP device /dev/media1 using PiSP variant BCM2712_C0

[0:25:06.994487330] [6792] WARN V4L2 v4l2_pixelformat.cpp:346 Unsupported V4L2 pixel format RPBP

[INFO] [1763728498.426692220] [camera]:

>> stream formats:

- *R8 (32x32 - 4608x2592)*
- *R16 (32x32 - 4608x2592)*
- *MONO_PISP_COMP1 (32x32 - 4608x2592)*
- *NV21 (32x32 - 4608x2592)*
- *SBGGR8 (32x32 - 4608x2592)*
- *SBGGR16 (32x32 - 4608x2592)*
- *BGGR_PISP_COMP1 (32x32 - 4608x2592)*
- *XBGR8888 (32x32 - 4608x2592)*
- *BGR888 (32x32 - 4608x2592)*
- *RGB888 (32x32 - 4608x2592)*
- *XRGB8888 (32x32 - 4608x2592)*
- *SGBRG16 (32x32 - 4608x2592)*
- *GBRG_PISP_COMP1 (32x32 - 4608x2592)*
- *SGRBG16 (32x32 - 4608x2592)*
- *GRBG_PISP_COMP1 (32x32 - 4608x2592)*
- *SRGGB16 (32x32 - 4608x2592)*
- *RGGB_PISP_COMP1 (32x32 - 4608x2592)*
- *BGR161616 (32x32 - 4608x2592)*
- *RGB161616 (32x32 - 4608x2592)*
- *SRGGB8 (32x32 - 4608x2592)*
- *SGRBG8 (32x32 - 4608x2592)*
- *SGBRG8 (32x32 - 4608x2592)*
- *YUYV (32x32 - 4608x2592)*
- *UYVY (32x32 - 4608x2592)*

[WARN] [1763728498.426919875] [camera]: no pixel format selected, auto-selecting: "XRGB8888"

[WARN] [1763728498.426952373] [camera]: set parameter 'format' to silence this warning

[INFO] [1763728498.427011685] [camera]:

>> XRGB8888 format sizes:

- *160x120*
- *240x160*
- *320x240*
- *400x240*
- *480x320*
- *640x360*
- *640x480*

- *720x480*
- *768x480*
- *854x480*
- *720x576*
- *800x600*
- *960x540*
- *1024x576*
- *960x640*
- *1024x600*
- *1024x768*
- *1280x720*
- *1152x864*
- *1280x800*
- *1360x768*
- *1366x768*
- *1440x900*
- *1280x1024*
- *1536x864*
- *1280x1080*
- *1600x900*
- *1400x1050*
- *1680x1050*
- *1600x1200*
- *1920x1080*
- *2048x1080*
- *1920x1200*
- *2160x1080*
- *2048x1152*
- *2560x1080*
- *2048x1536*
- *2560x1440*
- *2560x1600*
- *3840x1080*
- *2960x1440*
- *3440x1440*
- *2560x2048*
- *3200x1800*
- *3840x1600*
- *3200x2048*
- *3200x2400*
- *3840x2160*
- *4096x2160*

- 3840x2400

[WARN] [1763728498.427153067] [camera]: no dimensions selected, auto-selecting: "800x600"

[WARN] [1763728498.427177103] [camera]: set parameter 'width' or 'height' to silence this warning

[0:25:06.995539630] [6766] INFO Camera camera.cpp:1215 configuring streams: (0) 800x600-XRGB8888/sRGB

[0:25:06.995982016] [6792] INFO RPI pisp.cpp:1483 Sensor: /base/axi/pcie@120000/rp1/i2c@88000/imx708@1a - Selected sensor format: 1536x864-SBGGR10_1X10/RAW - Selected CFE format: 1536x864-PC1B/RAW

[INFO] [1763728498.428487928] [camera]: camera "/base/axi/pcie@120000/rp1/i2c@88000/imx708@1a" configured with 800x600-XRGB8888/sRGB stream

[INFO] [1763728498.428724490] [camera]: using default calibration URL

[INFO] [1763728498.428762636] [camera]: camera calibration URL: file:///home/slambot-rpi/.ros/camera_info/imx708__base_axi_pcie_120000_rp1_i2c_88000_imx708_1a_800x600.yaml

[ERROR] [1763728498.428864280] [camera_calibration_parsers]: Unable to open camera calibration file [/home/slambot-rpi/.ros/camera_info/imx708__base_axi_pcie_120000_rp1_i2c_88000_imx708_1a_800x600.yaml]

[WARN] [1763728498.428943276] [camera]: Camera calibration file /home/slambot-rpi/.ros/camera_info/imx708__base_axi_pcie_120000_rp1_i2c_88000_imx708_1a_800x600.yaml not found

[WARN] [1763728498.429400161] [camera]: unknown control: SyncFrames (20008)

[WARN] [1763728498.429813585] [camera]: unsupported control: AfWindows (37)

[WARN] [1763728498.430099478] [camera]: FrameDurationLimits: cannot set default scalar value '33333' on span control (extend: 2), default will be ignored

[WARN] [1763728498.430355651] [camera]: unknown control: SyncMode (20005)

[WARN] [1763728498.430634007] [camera]: unknown control: CnnEnableInputTensor (20011)

[WARN] [1763728498.430797073] [camera]: unknown control: NoiseReductionMode (10002)

[0:25:07.003304674] [6799] WARN IPARPI ipa_base.cpp:1399 Could not set AF_TRIGGER - no AF algorithm or not Auto

[0:25:07.003388689] [6799] WARN IPARPI ipa_base.cpp:1382 Could not set AF_PAUSE - no AF algorithm or not Continuous

[0:25:07.285421254] [6799] WARN IPARPI ipa_base.cpp:1399 Could not set AF_TRIGGER - no AF algorithm or not Auto

[0:25:07.285491861] [6799] WARN IPARPI ipa_base.cpp:1382 Could not set AF_PAUSE - no AF algorithm or not Continuous

The Good Stuff (Success Indicators)

- **Hardware Found:** Registered camera ... imx708
    - This confirms the Pi physically sees the camera and the libcamera overlay is working.
- **Pipeline Active:** configured with 800x600-XRGB8888/sRGB stream
    - This is the money line. The camera is actively capturing data and sending it to ROS.

The "Scary" Stuff (And why you can ignore it)

- **[WARN] ... no pixel format selected, auto-selecting**:
    - Because we didn't tell it what resolution to use, it guessed (800x600). This is fine for testing, but we will fix this with a launch file later.

- **[ERROR] ... Unable to open camera calibration file**:
    - **This is expected.** You haven't calibrated the camera yet. The node is looking for a file that defines how "curved" the lens is. Since it can't find it, it just outputs the raw image. This is perfectly fine for now.
- **[WARN] ... unknown control: SyncFrames / AfWindows**:
    - The ROS driver is trying to ask the camera for features (like specific autofocus zones) that the current Pi 5 driver doesn't expose in exactly the way ROS expects. These are harmless "noise" logs.

3. **View the feed:** On a separate computer (ensure both are on the same ROS_DOMAIN_ID and network), or in a new terminal on the Pi:

   ```
   ros2 run rqt_image_view rqt_image_view
   ```

   Select the topic /camera/image_raw (or /camera/image_compressed if viewing over WiFi to reduce lag) in the top left corner and you should see your video feed!

Troubleshooting

- **"No cameras available"**: Double-check your ribbon cable. The text on the cable should face **towards** the HDMI ports on the Pi 5.
- **Permission Denied**: Ensure your user is in the video group: sudo usermod -aG video $USER, then log out and back in.
- **Shared Library Error**: If it complains about missing libraries, try exporting the path manually before running: export LD_LIBRARY_PATH=/usr/local/lib/aarch64-linux-gnu:$LD_LIBRARY_PATH

Final thing, to ensure you can latch onto a compressed image feed (for Dev Machine RVIZ purposes – otherwise the video feed is way too slow to be useful) make sure you have the following installed on BOTH the RaspberryPi and the DevMachine:

```
sudo apt update
sudo apt install ros-jazzy-image-transport-plugins
```